

Wochenprotokoll: VisionQuest

Name: Matthias

Datum: 06.03.2026

Meilenstein: M6 + M7 + M8

Projektstatus:  Im Plan /  Verzögert /  Kritisch

1. Was wurde diese Woche gemacht?

- Alle 6 Screens vollständig implementiert und verbunden
- Routing & Navigation zentralisiert und stabil integriert
- Responsive Layouts für Mobile/Tablet/Desktop umgesetzt
- Riverpod State Management für globalen AppState integriert
- 5 Themes implementiert (Light, Dark, System, RetroArcade, AdventureMap)
- Quest-Logik (XP, Level, Streak) eingebaut und UI-seitig angebunden
- Animationen (Rive/Lottie) in Reward-/UI-Flow eingebaut
- Code Cleanup durchgeführt (Struktur, Lesbarkeit, Konsistenz verbessert)

2. Wie liegt man im Zeitplan?

- Im Zeitplan - M7-Ziele für UI/Navigation und State/Theme sind abgeschlossen
- App-Navigation und responsives Verhalten laufen stabil auf den Zielgrößen
- Animationen sind integriert und verbessern die UX ohne funktionale Regressionen
- Code Cleanup ist umgesetzt, Wartbarkeit und Konsistenz wurden verbessert

3. Was sind die nächsten Schritte?

- Finale Dokumentation abschließen
- Projektabgabe vorbereiten und einreichen

4. KI-Einsatz & Dokumentation

Modell: GitHub Copilot (GPT-5.3-Codex)

1. Zentrales Routing in Flutter

Prompt: "Wie baue ich in Flutter ein sauberes Named-Routing mit zentraler Route-Definition auf?"

Ergebnis: Klare AppRoutes-Struktur mit zentraler Route-Map und argumentbasierten Übergaben

Integration: Navigation wurde auf zentrale Routen umgestellt; alle Screens sind konsistent angebunden und leichter wartbar.

2. Responsive Screen-Layouts

Prompt: "Wie gestalte ich Flutter Screens responsiv für Handy, Tablet und Desktop ohne doppelten Code?"

Ergebnis: LayoutBuilder/ConstrainedBox-Ansatz mit Breakpoints und dynamischen Breiten/Paddings

Integration: Relevante Screens wurden auf adaptive Layouts umgebaut; Inhalte skalieren sauber über verschiedene Auflösungen.

3. Riverpod State Management

Prompt: "Wie strukturiere ich globalen AppState mit Riverpod für Theme, Progress und Quest-Log?"

Ergebnis: StateNotifier + immutable State-Modelle für zentrale, nachvollziehbare Updates

Integration: Theme, XP/Level/Streak und Log-Einträge laufen über einen zentralen Provider; UI reagiert direkt per ref.watch().

4. Theme-System mit 5 Varianten

Prompt: "Wie implementiere ich mehrere Theme-Varianten inkl. System-Mode in Flutter sauber?"

Ergebnis: Theme-Enums + zentrale ThemeFactory mit ThemeData/ThemeMode-Mapping

Integration: Nutzer kann zwischen 5 Themes wechseln; die Auswahl wirkt appweit und konsistent auf alle Screens.

5. Quest-Logik (XP, Level, Streak)

Prompt: "Wie berechne ich XP, Level und Daily-Streak robust und ohne Doppelzählung?"

Ergebnis: Definierte Berechnungsregeln inkl. Tagesgrenzen, Streak-Fortsetzung und Reset-Logik

Integration: Bei Quest-Abschluss werden XP, Level und Streak korrekt aktualisiert; die Werte werden direkt im Home- und Log-Bereich angezeigt.

6. Animationen (Lottie/Rive)

Prompt: "Wie integriere ich Lottie/Rive performant in Flutter-Screens mit Fallback bei Ladefehlern?"

Ergebnis: Saubere Einbindung mit Lade-/Fehlerzuständen und stabiler Screen-Integration

Integration: Reward- und UI-Übergänge wurden mit Animationen ergänzt und UX verbessert.

7. Code Cleanup

Prompt: "Wie führe ich in Flutter ein sicheres Code Cleanup durch, ohne Verhalten zu verändern?"

Ergebnis: Refactoring-Leitlinien für Struktur, Benennung, Redundanzabbau und kleine API-Bereinigungen

Integration: Cleanup wurde auf zentrale Module angewendet, Lesbarkeit und Wartbarkeit verbessert.